



FIREBRAND

LYCEUM EDUCATION GROUP

App Structure and Reusable Components

Power Platform Pathway | Session 5

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: App Structure and Reusable Components

Lesson Objectives

Apply

- Apply modular design principles in Power Apps

Analyse

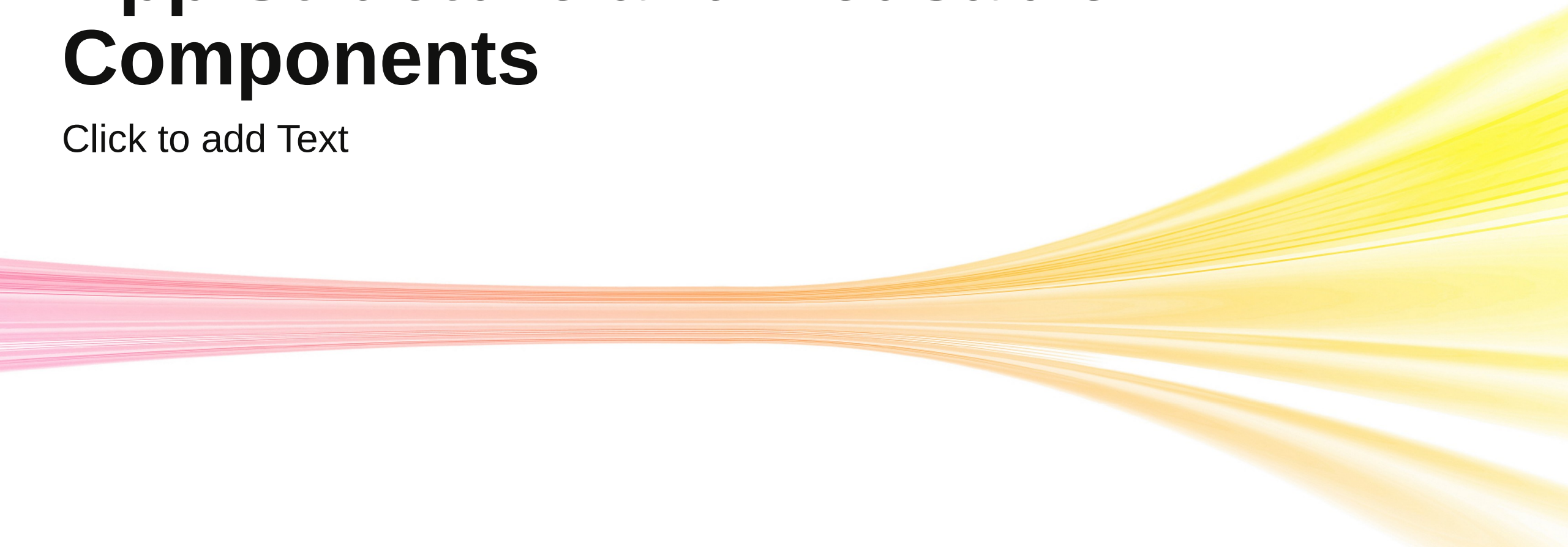
- Analyse app structure for inefficiencies

Evaluate

- Create reusable components and scalable layouts

App Structure and Reusable Components

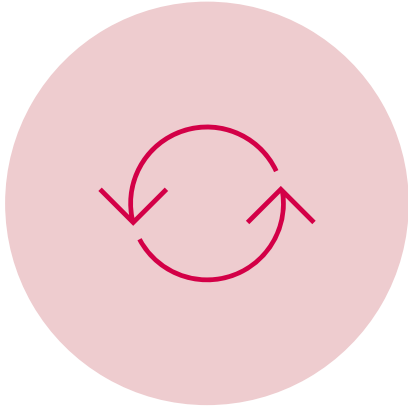
Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Why Build Components?

- A component packages a group of controls and logic into a single reusable unit
- Reduces duplication across screens and keeps design consistent

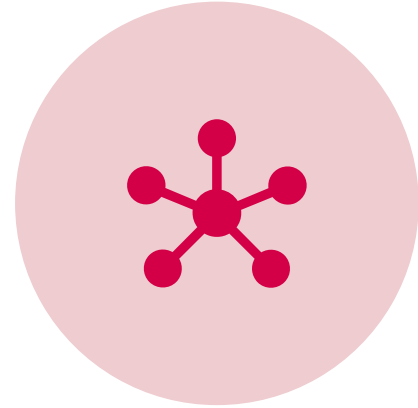
Core Concepts



REUSABLE UNITS



**WRITE ONCE,
INSTANTIATE
EVERYWHERE**



**CENTRALISED
MAINTENANCE**

Input vs. Output Properties

// Incoming Input Property Mapping:

// cmp_CustomTaxCalculator.InvoiceAmount
(Input, Number)

// Outgoing Output Property Return Formula:

// Assign this directly inside the component's
output property definition:

cmp_CustomTaxCalculator.InvoiceAmount * 0.20

The Data Flow:

- **Input Properties (Parameters):** Variables that accept incoming data strings or objects from the active screen into the component
- **Output Properties (Return Values):** Computed states or user-selected records sent directly back to the screen's main execution loop
- **The Power Fx Analogy:** Think of Input Properties as a method's configuration flags, and Output Properties as its structural return calculation

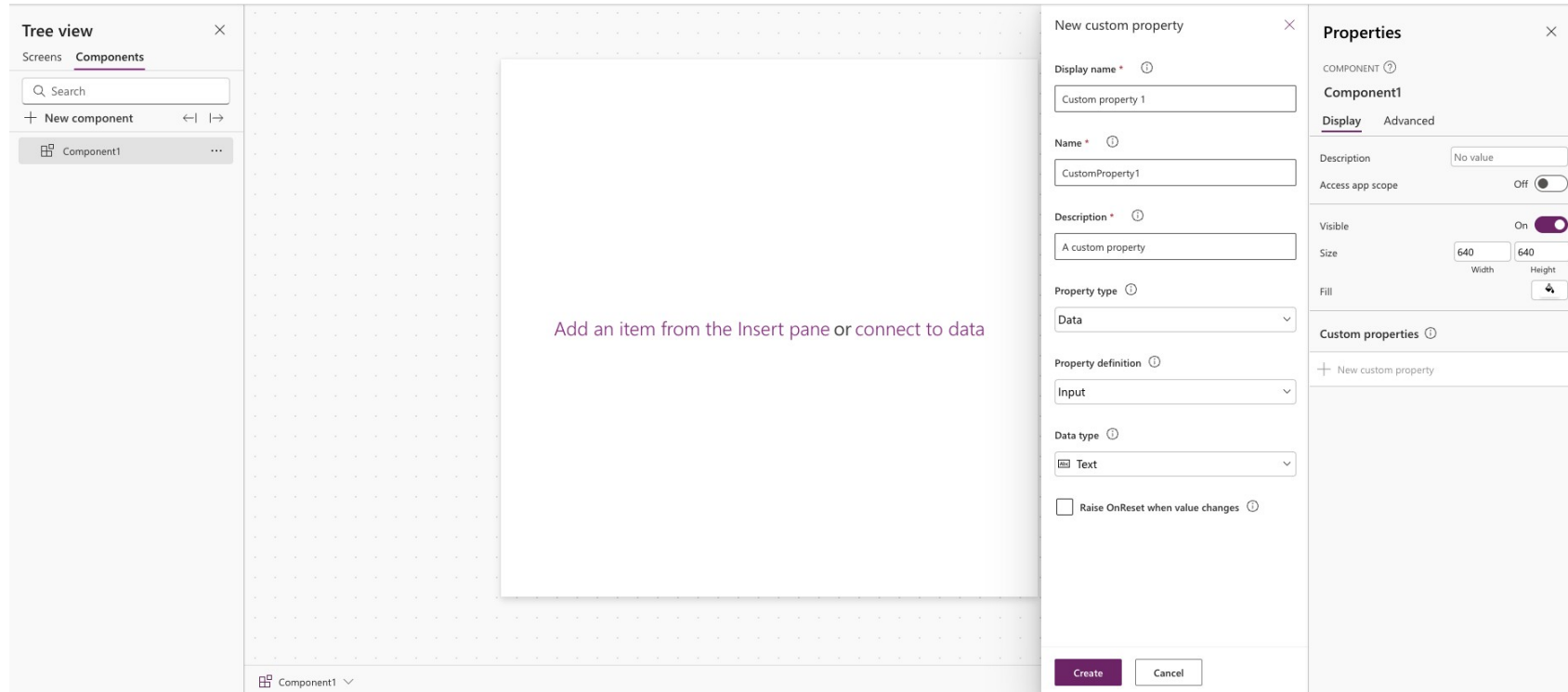
Component Input Properties

Custom properties let a component behave differently depending on where it's used

// Custom property: LabelText (Input, Text)

// Used inside the component

Label1.Text = LabelText



Access App Scope

By default, canvas components live in a strict security sandbox

They are completely blind to global variables, collections, and formulas created in the host app

Enabling Access

- To map central tables or user data into your component, select the main component frame in the Tree View and toggle "**Access App Scope**" to **ON** in the right-hand properties panel

Production Rule

- Turn this on **only** for dedicated, app-specific utilities like sidebar navigation templates. Leave it off for completely generic, multi-app UI tools like generic buttons or custom sliders

Access App Scope: OFF	Access App Scope: ON
Component Cannot Read: • Global Variables (Set) • App Collections • Tabular Data Sources	Component Direct Access to: • Global Variables (Set) • Central App Collections • Dataverse & SharePoint Tables

State-Based UI Binding

A reusable navigation sidebar component (cmp_SideNav) doesn't natively know which app screen it is currently sitting on. Without this context, you cannot visually highlight the active menu choice for the user journey

The Solution: Screen Binding via Custom Inputs

- **Step 1:** Inside your component settings, create a **New Custom Property**.
- **Step 2:** Name it CurrentHostScreen, set the Property Type to **Input**, and select the Data Type as 'Screen'
- **Step 3:** When dropping the component onto a canvas screen (e.g., Screen_Dashboard), bind the property explicitly

Powerapps:

```
cmp_SideNav_Instance.CurrentHostScreen = App.ActiveScreen
```

Driving Visual Active States

Inside the component's menu gallery template, select the item background box or icon graphic, and set its Fill or Color property to dynamically toggle brightness based on the matched location:

```
If(  
    ThisItem.TargetScreen = Parent.CurrentHostScreen,  
    Color.FromHex("#E21A42"), // Active Theme Color  
    Color.Transparent         // Default State  
)
```

Declarative Logic via App.Formulas

Legacy Power Apps reliance on App.OnStart utilises Procedural Logic- using commands like Set() and ClearCollect() to explicitly write state details to memory over time

Modern modular design leverages Declarative Logic via App.Formulas

Code Implementation Setup

- Instead of setting global app menus inside your boot configurations, write them cleanly as a standard, fixed data schema blueprint under the main App.Formulas option:

```
FX_NavigationMenu = Table(  
    { Title: "Dashboard", Icon: Icon.TrendingUp, TargetScreen: Screen_Dashboard },  
    { Title: "New Order", Icon: Icon.Add, TargetScreen: Screen_NewOrder },  
    { Title: "Settings", Icon: Icon.Settings, TargetScreen: Screen_Settings }  
);
```

Key Operational Distinctions

Metric	Legacy (App.OnStart)	Modern (App.Formulas)
Execution	Runs strictly once sequentially when the application boots	Calculates instantly on-demand and caches the data structure
Mutability	Variables are mutable; values can be accidentally changed anywhere	Read-Only constants. The value is fully protected
Component Use	Requires passing deep parameters across sandbox links	Easily read globally via App Scope

Monolithic vs. Modular Components

Duplicated Across Screens:

// THE MONOLITHIC MESS (Duplicated on Button1, Button2, and Button3)

```
Set(varDiscount, txt_Price.Text * 0.10);  
Set(varTax, (txt_Price.Text - varDiscount) * 0.20);  
Set(varTotal, txt_Price.Text - varDiscount + varTax)
```

Reusable Component:

// THE MODULAR COMPONENT SOLUTION (Single Global Call)

// Point your UI output label property straight to the component execution hook:

```
cmp_FinancialEngineInstance.TotalComputedAmount
```

Why Component Isolation Wins

- **Readability Boost:**

Abstracts messy formula brackets away from the main canvas screens, leaving a self-documenting semantic tag layout

- **Rapid Code Reuse:**

Run identical business calculations across multiple screens instantly without creating complex global variable webs

- **Team Scalability:**

Multiple developers can safely build separate components or custom property functions simultaneously without causing screen git conflicts

Scalable Layouts & Relative Positioning



Hardcoding fixed positions (e.g., Width = 1366, X = 200) ruins app scalability

If a user opens the app on a tablet, a phone, or a high-res monitor, elements will clip, overlap, or leave massive empty spaces

The Modular Solution: Dynamic Coordination

- Instead of typing static numbers into control properties, write expressions relative to the parent screen or surrounding components using the **Parent** operator:

// Make the header dynamically span any screen width perfectly

Parent.Width

- **Dynamic Main Content X-Coordinate** (*shifting a gallery over to accommodate the sidebar menu*):

// Positions the content exactly where the sidebar menu ends

cmp_SideNav_Instance.X +
cmp_SideNav_Instance.Width + 20

Enterprise Best Practice

Layout Containers;

- Insert a native **Vertical or Horizontal Container** control directly onto the canvas
- Drop your components and forms inside it
- Turn on "**Flexible Width/Height**" in the properties panel to let Power Apps auto-calculate alignment, padding, and spacing uniformly across all screens

Navigation Architecture

- **The Navigate() Blueprint:** Changes the viewable screen buffer by targeting an explicit screen object, running animations natively on the device GPU
- **Contextual Parameter Passing:** You can inject hidden, temporary variables directly inside the navigation pipeline instead of hardcoding heavy global states
- **The Memory Cycle:** Use **Back()** to automatically step down the runtime history stack, which clears unneeded screen states out of temporary hardware memory

Production Code Example:

// Standard Animated Transaction

```
Navigate(ScreenProductList, ScreenTransition.Fade)
```

// Advanced Context-Driven Navigation (Passes a record reference forward)

```
Navigate(ScreenDetailView, ScreenTransition.Cover,  
{ SelectedItemRecord: ThisItem })
```

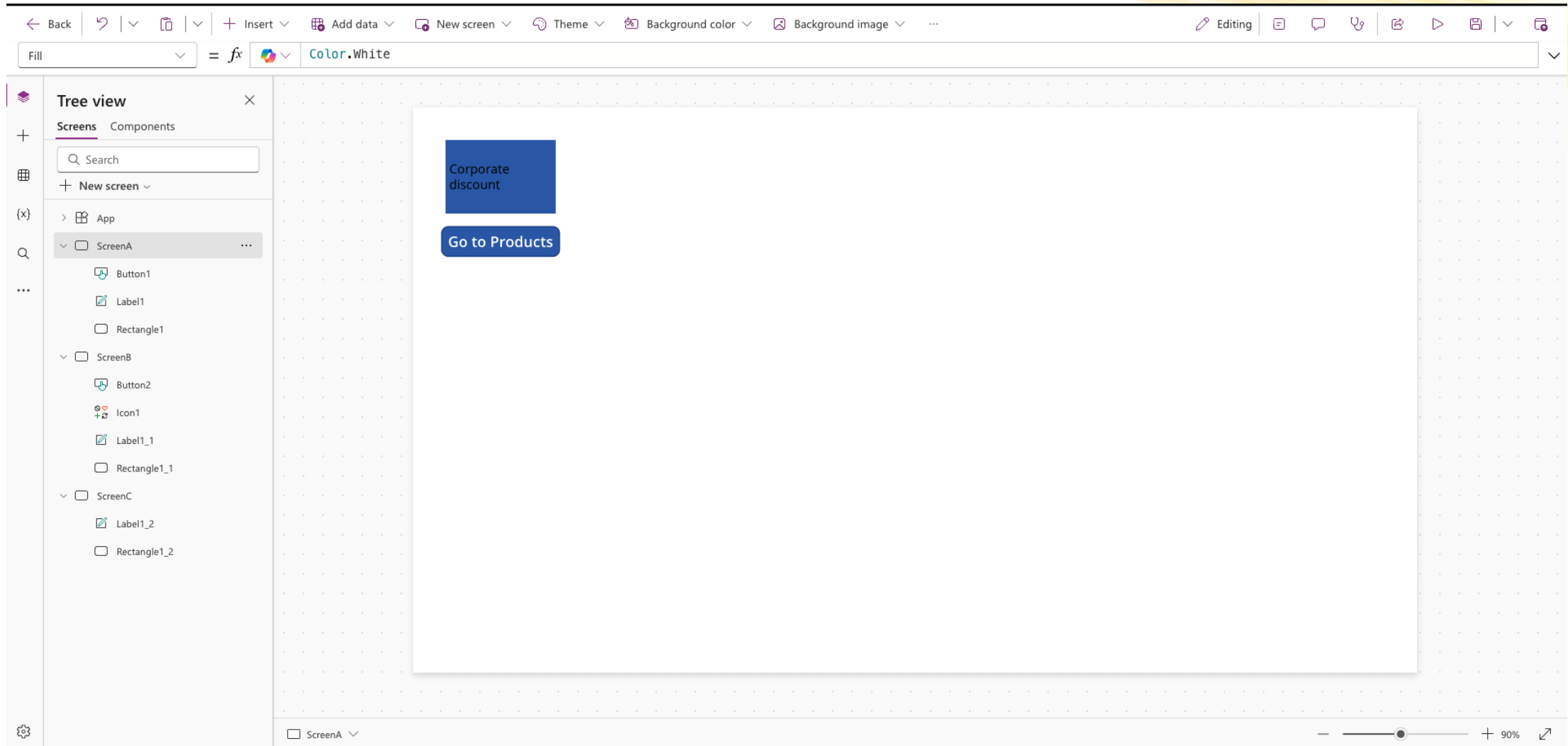
App Analysis Activity

Objective: Analyse the unstructured repository file **M5S5_Baseline_App.msapp** to run a technical design review before coding

The 3-Point Component Audit Framework:

- **Visual Duplication Scan:** Pinpoint repeating control clusters (e.g., identical header bars, navigation menus) built manually on separate canvas layout screens
- **Transactional Path Trace:** Track every single button containing a raw, un-parameterized Navigate() loop. Flag occurrences where clicking back gets users lost in loop errors
- **Variable Sprawl Check:** Count instances where global states (Set) are used to awkwardly force matching screens to behave consistently

Messy Tree View



Refactoring Workshop

Objective:

- Improve the provided app by creating reusable components for common functionality

Tasks:

- Organise screens into a clear and logical structure
- Standardise formulas, controls and naming conventions
- Test the changes to confirm functionality remains consistent

Reusable Component Library



ACTIVITY



Objective: Design and build reusable app components

Tasks / Requirements:

- Create an input form component
- Create reusable navigation controls
- Apply components throughout the app to reduce duplication

Key Point: Components must function consistently wherever they are used

Extension Activity: Create additional reusable components such as a notification banner



ACTIVITY



Peer Review Activity

- Provide structured feedback
- Identify opportunities to improve navigation and user experience
- Suggest enhancements to component design and app organisation

Design Discussion

Learners discuss and compare:

- Collaboration within development teams
 - Maintainability of applications
 - Faster development and deployment cycles
 - Consistency across larger solutions
-
- **Expected Outcome:** Learners can explain how modular design contributes to efficient development and long-term maintainability

Stretch and Challenge

- Design a scalable multi-screen application structure
- Apply reusable components across multiple areas of the solution
- Justify navigation, layout and component design decisions

Session Summary

Click to add Text

